

ARCHITECTURE DOCUMENT

Sales Eyes

A simple, visual guide to how the app is built and how information flows through it — from a blank prospect search to a ready-to-use sales script.

Next.js Frontend

FastAPI Backend

PostgreSQL

AI-Powered

1

The Big Picture

WHAT SALES EYES ACTUALLY DOES

Sales Eyes helps a salesperson walk into a conversation prepared. You type in whatever you already know about a prospect — a company name, a person's role, a LinkedIn link, anything — and the app does the legwork: it researches the company and contact online, organizes what it finds, and turns that into a tailored sales script.

Think of it like this: you hand the app a name, and it hands you back a research dossier plus a script written specifically for that prospect — ready to use in your next call.

R

Research

Searches the web, reads pages, and pulls out useful facts about the company and contact.

M

Make Sense of It

Cleans up conflicting or duplicate info and scores how confident it is in each fact.

W

Write the Script

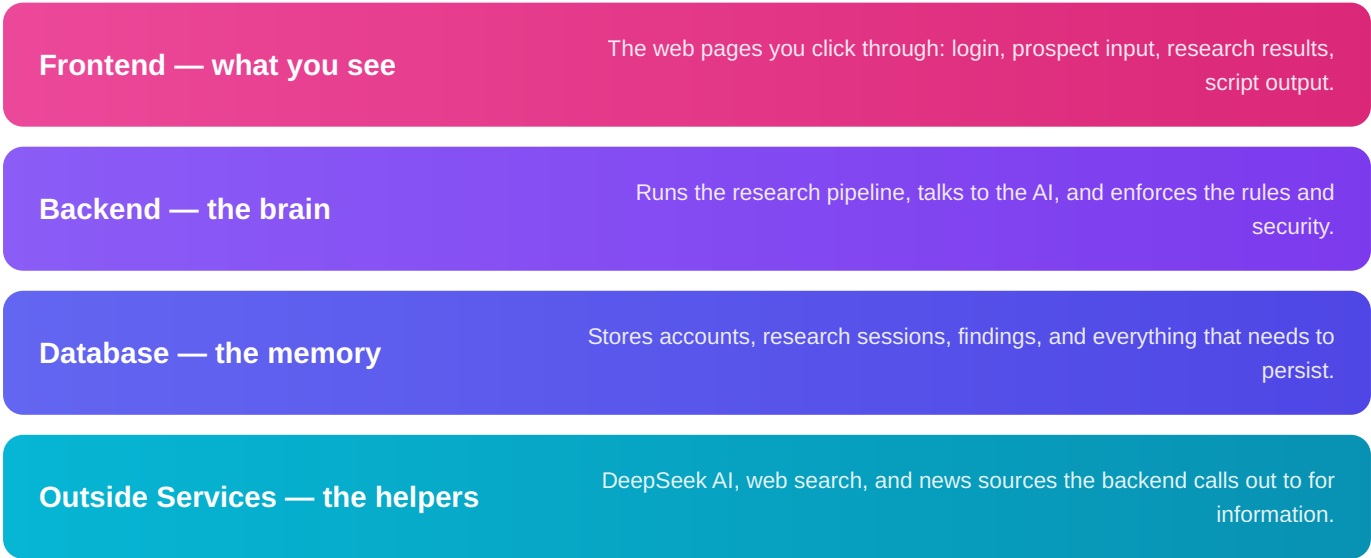
Uses AI to turn the best findings into a sales script, in the style you choose.

2

The Three Building Blocks

HOW THE PIECES FIT TOGETHER

Under the hood, Sales Eyes is split into three simple layers that each do one job well. This separation keeps the app easy to understand, test, and extend.

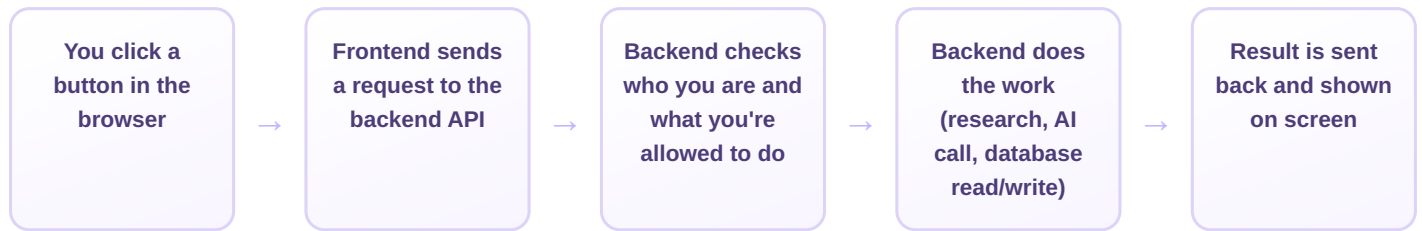


3

How a Request Flows

FROM CLICK TO SCRIPT

Every action in Sales Eyes follows the same basic path. Here's what happens, step by step, when you use the app:



Why this matters: the frontend never talks to the database or the AI directly — it always goes through the backend. That keeps your data and API keys safe, and keeps the rules in one place.

4

The Research Journey

WHAT HAPPENS AFTER YOU HIT "START RESEARCH"

- 1 You describe the prospect**
Type in anything you know — a company, a name, a hint. No rigid form required.
- 2 The app builds a research plan**
AI turns your notes into a checklist of things worth researching, like "Company background" or "Recent news."
- 3 The research pipeline runs**
The backend searches the web, fetches pages, and pulls out facts using its research "providers."
- 4 Findings are cleaned up**
If two sources disagree, the Intelligence engine flags the conflict and scores how trustworthy each finding is.
- 5 You pick what to use**
You review the findings and select the ones worth building a pitch around.
- 6 You choose a sales style**
SPIN, Challenger, Sandler, or your own custom approach.
- 7 The script is generated**
DeepSeek AI combines your selected findings and chosen style into a ready-to-use script.

5

Inside the Frontend

NEXT.JS + REACT + TAILWIND

The frontend is what the user actually sees and clicks. It's built with Next.js (a React framework) and styled with Tailwind CSS. Each screen is its own "page":

Screen	What it's for
Login / Register	Create an account and sign in — protected by a login token (JWT)
Homepage	The freeform box where you describe a prospect and kick off research
Dashboard	Lists your past and current research sessions
Research Session	Shows the research plan, findings, and lets you select what to use
Intelligence View	Deeper look at candidate matches, conflicting facts, and confidence scores
Settings	Where you can plug in your own AI API key at runtime

6

Inside the Backend

FASTAPI, ORGANIZED BY RESPONSIBILITY

The backend is written in Python with FastAPI. It's organized so each folder has one clear job — this makes it much easier to find and change things without breaking something else.

routes/

The "front doors" of the API — one file per feature area: auth, research, materials, intelligence. Each route just receives a request and hands it off to a service.

services/

Where the real logic lives — talking to DeepSeek, running the research pipeline, generating scripts, handling pain-point detection.

models/

Defines the shape of the data as it's stored in the database (users, sessions, findings, materials).

core/

Shared plumbing: configuration, database connection, security/authentication rules.

Simple rule of thumb: routes decide "who can ask for what," services decide "how to actually do it," and models decide "what the data looks like."

7

The Intelligence Engine

HOW SALES EYES TELLS FACT FROM NOISE

When you research a real company, the internet gives you messy, sometimes contradictory information. The Intelligence engine is the part of the backend that turns that mess into something trustworthy.

S

Search & Fetch

Finds relevant pages using web search, then downloads and reads them.

E

Extract

Pulls out names, roles, companies, and other useful entities from the raw text.

I

Identity Resolver

Figures out if two mentions refer to the same real person or company.

C

Conflict Detector

Flags when sources disagree, e.g. two different job titles for the same person.

S

Confidence Scorer

Rates how trustworthy each finding is, based on source quality and agreement.

O

Orchestrator

Runs all the above in the right order and tracks the job until it's done.

In plain words: the engine doesn't just dump raw search results on you — it cross-checks facts, tells you when something is uncertain, and shows its work through an evidence timeline.

8

The Database

WHAT GETS REMEMBERED, AND WHERE

Sales Eyes uses PostgreSQL to store everything that needs to persist between visits. The schema grows through small, numbered migration files, so changes are tracked over time.

Data	What it holds
Users	Accounts and login credentials
Research Sessions	Each prospect research effort, and its plan
Findings	Individual facts discovered during research
Materials	Product PDFs/Word docs you've uploaded for context
Intelligence Data	Candidates, conflicts, and confidence scores

9

How It Runs

DOCKER COMPOSE TIES IT ALL TOGETHER

All three pieces — frontend, backend, and database — run as separate Docker containers, wired together by `docker-compose.yml`. That means one command starts the whole app on your machine:

```
# one command starts everything
docker-compose up

# what gets started:
frontend    → Next.js app           → http://localhost:3000
backend     → FastAPI app           → http://localhost:8000
database    → PostgreSQL            → localhost:5432
```

The backend also talks out to a couple of external services over the internet — DeepSeek for AI generation, plus optional web search and news APIs for research.

10

Folder Map

WHERE TO FIND THINGS

```
sales-eyes/
├── frontend/      # Next.js pages & components
│   ├── app/       # screens: login, dashboard, research...
│   └── components/ # reusable UI pieces
├── backend/       # FastAPI application
│   ├── app/
│   │   ├── routes/ # API endpoints
│   │   ├── services/ # business logic & AI calls
│   │   ├── models/  # database table definitions
│   │   └── core/    # config, security, db connection
├── database/      # SQL migration files
└── docker-compose.yml # starts the whole stack
```

Takeaway: Sales Eyes is deliberately simple to reason about — a screen layer, a logic layer, and a storage layer, connected by clear API calls, with an AI-powered research engine doing the heavy lifting in the middle.